

Mini Git

평가자 질문과 답변

20개 질문 · 쉬운 답변 · 코드 근거 · 개선 제안

기능 확인 → 구조 설명 → 원리 이해 → 개선 제안

항목 1: 6개 질문 항목 2: 5개 질문
항목 3: 5개 질문 항목 4: 4개 질문

항목 4는 현재 미구현인 개선 전략입니다.

INIT은 새 저장소의 시작

평가자 질문

INIT <user_name> 실행 후 main 브랜치/HEAD/현재 사용자 설정이 정상적으로 초기화되어 있는가?

쉬운 답변 · 현재 구현

INIT은 기존 커밋과 검색 색인을 비우고 main 브랜치를 만듭니다. 현재 브랜치는 main, 현재 사용자는 입력한 이름으로 설정됩니다. 첫 커밋 전에는 main이 가리키는 커밋이 없어 HEAD 값은 None입니다.

예시 · 확인할 점

```
mini-git>에서 입력  
INIT "Alice Kim"  
LOG
```

```
확인: 아직 커밋 없음.  
current_branch = "main"  
branches["main"] = None  
current_user = "Alice Kim"
```

INIT은 이름만 바꾸는 명령이 아니라 저장소를 새로 시작하는 명령입니다.

브랜치는 커밋을 가리키는 이름표

평가자 질문

BRANCH <name> 생성 후 SWITCH <name>로 전환되며, 이후 COMMIT이 해당 브랜치에 반영되는가?

쉬운 답변 · 현재 구현

BRANCH는 현재 커밋을 가리키는 새 이름표를 만듭니다. BRANCH만으로 현재 브랜치가 바뀌지는 않습니다.

SWITCH로 전환한 뒤 COMMIT하면 새 커밋의 부모는 그 브랜치의 마지막 커밋이 되고, 해당 브랜치만 새 커밋으로 이동합니다.

예시 · 확인할 점

```
mini-git>에서 입력  
INIT "Alice Kim"  
COMMIT "Initial commit"  
BRANCH feature  
SWITCH feature  
COMMIT "Add login feature"
```

```
main → 최초 커밋 A  
feature → 새 커밋 B  
B의 부모 → A
```

BRANCH는 생성, SWITCH는 전환, COMMIT은 현재 브랜치 갱신입니다.

LOG는 부모를 먼저 보여줍니다

평가자 질문

LOG가 “부모 커밋이 자식 커밋보다 먼저” 출력되도록 동작하는가?

쉬운 답변 · 현재 구현

LOG는 날짜가 아니라 부모 관계를 기준으로 순서를 정합니다. 부모가 없는 커밋부터 출력하고, 아직 출력하지 않은 부모가 0개인 자식을 다음 후보에 넣습니다.

이 방식은 Kahn의 위상 정렬입니다. 동시에 후보가 된 커밋은 큐에 들어온 순서로 처리합니다.

예시 · 확인할 점

부모 참조
 $B \rightarrow A, C \rightarrow A$

LOG의 가능한 출력
 $A \rightarrow B \rightarrow C$

핵심: A는 반드시 B·C 앞.
B와 C의 선후는 부모 조건이 아님.

검증: 각 부모의 출력 위치가
자식의 위치보다 앞인지 확인.

부모가 먼저라는 규칙을 날짜가 아닌 그래프 관계로 보장합니다.

PATH는 가장 짧은 연결을 찾습니다

평가자 질문

PATH <a> 가 경로가 있으면 최단 경로를, 없으면 No path를 출력하는가?

쉬운 답변 · 현재 구현

PATH는 부모·자식 연결을 양방향으로 보고 BFS로 최단 거리를 찾습니다. 출발점과 도착점에서 각각 거리를 구하고, 최단 경로 위의 다음 후보 중 hash가 가장 작은 것을 선택합니다.

연결이 없으면 No path를 출력합니다. 존재하지 않는 hash는 Unknown commit 오류로 구분합니다.

예시 · 확인할 점

B의 부모 A, C의 부모 A
PATH B C → B → A → C
간선 수는 2개입니다.

독립된 두 루트 A와 X
PATH A X → No path

PATH A A → A
자기 자신까지의 간선 수는 0.
※ A/B/C는 설명용 표식입니다.

거리부터 최소화하고, 거리가 같은 경로는 hash 사전순으로 고릅니다.

ANCESTORS는 모든 조상을 모읍니다

평가자 질문

ANCESTORS <hash>가 모든 조상을 빠짐없이 출력하는가?

쉬운 답변 · 현재 구현

부모 목록부터 시작해 부모의 부모를 BFS로 따라갑니다. 이미 방문한 hash는 set에 기록해 다시 결과에 넣지 않습니다.

두 경로 중 하나만 고르는 것이 아니라 모든 조상을 모읍니다. 시작 커밋 자신은 결과에서 제외합니다.

예시 · 확인할 점

D의 부모: B, C

B의 부모: A

C의 부모: A

ANCESTORS D → B, C, A

A는 두 경로로 만나도 한 번만.

D는 자기 자신이므로 제외.

부모 없는 A → No ancestors.

부모를 모두 따라가되, 같은 조상은 한 번만 기록합니다.

검색과 정렬은 기준이 다릅니다

평가자 질문

SEARCH <keyword> / SEARCH --author=<name> / LOG --sort-by=date|author가 요구사항대로 동작하는가?

쉬운 답변 · 현재 구현

SEARCH는 단어·작성자 역색인으로 커밋을 찾습니다. 단어는 소문자로 통일하고, 여러 단어는 순서대로 붙어 있는 구문인지 검사합니다. 작성자 이름은 전체가 정확히 일치해야 합니다.

LOG --sort-by는 직접 만든 안정 병합 정렬을 사용합니다. 날짜 또는 작성자 기준이며, 일반 LOG의 부모 우선 규칙과는 별도입니다.

예시 · 확인할 점

메시지: Add login feature
SEARCH login → 일치
SEARCH "login feature" → 일치
SEARCH "feature login" → 불일치

SEARCH --author="Alice Kim"
LOG --sort-by=date
LOG --sort-by=author

CLI 작성자는 INIT에서 정합니다.

검색은 조건에 맞는 커밋을 찾고, 정렬은 출력 순서를 바꿉니다.

저장 상태와 알고리즘을 분리했습니다

평가자 질문

커밋 저장소/브랜치/HEAD/사용자 정보를 어떤 구조로 분리했고, 각 책임을 설명할 수 있는가?

쉬운 답변 · 현재 구현

Repository.commits는 hash → Commit 디렉터리입니다. branches는 브랜치 이름 → 마지막 커밋 hash, current_branch는 현재 브랜치 이름, current_user는 작성자를 저장합니다.

현재 HEAD는 branches[current_branch]로 확인합니다. CLI는 입력·출력, Repository는 상태, 별도 모듈은 알고리즘을 담당합니다.

예시 · 확인할 점

main.py → 실행 시작
cli.py → 명령 해석·결과 출력
repository.py → 저장 상태 관리
commit.py → 커밋·hash 생성
traversal.py → 그래프 탐색
sorting.py → 병합 정렬
index.py → 검색 색인
errors.py → 오류 종류

입력, 상태, 알고리즘을 나누면 각 부분을 따로 이해하고 검사할 수 있습니다.

hash로 바로 찾고 중복은 재시도합니다

평가자 질문

커밋 hash로 빠르게 조회하기 위해 어떤 키-값 구조를 사용했고, 중복/충돌을 어떻게 방지했는지 설명할 수 있는가?

쉬운 답변 · 현재 구현

dict는 hash를 키로 사용해 커밋을 평균 $O(1)$ 에 조회합니다. list에서 같은 hash를 찾으면 하나씩 확인해야 하므로 $O(N)$ 입니다.

메시지·작성자·시각·부모·순번을 SHA-1으로 계산해 앞 6자리를 사용합니다. 기존 hash와 같으면 순번을 바꿔 다시 생성합니다. 순번만으로 충돌이 사라지는 것은 아닙니다.

예시 · 확인할 점

`commits["5f8188"]` → 해당 커밋

hash 생성 → 이미 있는가?

아니요 → 사용

예 → 새 순번으로 다시 생성

6자리의 가능한 값: 16^6 개

= 16,777,216개

공간이 다 차면 현재 재시도는 종료되지 않는 한계가 있습니다.

중복을 막는 핵심은 기존 hash 검사와 충돌 시 재시도입니다.

COMMIT 직후 검색 색인을 갱신합니다

평가자 질문

커밋이 추가될 때 역색인(author/keyword)을 어떤 시점에, 어떤 방식으로 갱신하도록 설계했는지 설명할 수 있는가?

쉬운 답변 · 현재 구현

새 Commit을 저장하고 현재 브랜치를 갱신한 다음 index.add_commit을 호출합니다. 따라서 새 커밋을 바로 검색할 수 있습니다.

메시지를 공백으로 나누고 소문자로 바꿉니다. 단어 → hash 목록과 작성자 → hash 목록에 추가하며, 같은 커밋의 반복 단어는 한 번만 색인합니다.

예시 · 확인할 점

COMMIT "Login login feature"

토큰: login, login, feature

단어별 중복 제거

login → [새 hash] 한 번만 추가

feature → [새 hash]

Alice Kim → [새 hash]

INIT을 다시 실행하면

두 색인도 새로 비웁니다.

커밋을 만드는 순간 색인도 갱신하므로 검색할 때 전체를 훑지 않습니다.

탐색 함수를 따로 호출할 수 있습니다

평가자 질문

LOG, PATH, ANCESTORS에서 사용하는 그래프 탐색 로직을 어떻게 재사용 가능하게 구성했는지 설명할 수 있는가?

쉬운 답변 · 현재 구현

LOG·PATH·ANCESTORS의 탐색은 traversal.py의 독립된 함수입니다. 커밋 디렉터리와 필요한 hash를 입력받아 결과를 반환하고, 화면 출력이나 현재 사용자 상태에 의존하지 않습니다. PATH는 양쪽 거리 계산에 같은 BFS 함수를 재사용합니다. 조상 탐색은 부모 방향만 이동하는 별도 함수로 구현합니다.

예시 · 확인할 점

```
topological_order(commits)
ancestors(commits, hash)
shortest_path(commits, a, b)
```

Repository → 함수 호출
함수 → 결과 반환
CLI → 결과 출력

테스트도 CLI 없이 직접 호출해
다중 부모 그래프를 검사합니다.

저장 상태와 화면에 의존하지 않는 함수로 탐색을 재사용합니다.

주석에는 동작과 이유를 적었습니다

평가자 질문

주요 함수/클래스에 docstring/주석을 어떤 기준으로 작성했는지 설명할 수 있는가?

쉬운 답변 · 현재 구현

클래스에는 역할과 주요 필드, 함수에는 입력·반환값·처리 규칙을 설명했습니다. 단순히 코드 한 줄을 되풀이하기보다 선택한 이유와 주의할 조건을 적었습니다.

예를 들어 부모 방향, 방문 중복 제거, hash 충돌 재시도, <=로 안정성을 유지하는 이유, 시간복잡도를 설명합니다.

예시 · 확인할 점

Commit: hash와 parents의 의미
ancestors: 자신 제외·중복 제거
shortest_path: 무방향·사전순
_merge: 같으면 왼쪽 먼저
generate_commit_hash:
기존 hash와 비교 후 재시도

원칙: 무엇을 하는가 + 왜 그렇게 하는가 + 어떤 조건이 중요한가.

다음 사람이 코드의 목적과 선택 이유를 이해할 수 있게 설명했습니다.

DAG는 방향이 있고 순환이 없습니다

평가자 질문

커밋 그래프가 왜 DAG여야 하는지, 사이클이 생기면 어떤 문제가 발생하는지 설명할 수 있는가?

쉬운 답변 · 현재 구현

커밋은 부모를 가리키므로 방향이 있습니다. 새 커밋은 먼저 존재하던 커밋만 부모로 참조하고 기존 부모 참조를 바꾸지 않습니다. 따라서 부모를 따라가면 생성 순서가 계속 이전으로 가며 자기 자신으로 돌아올 수 없습니다.

사이클이 생기면 부모 우선 순서는 만들 수 없습니다. 현재 Kahn 구현은 순환 노드를 모두 출력하지 못하며, 별도 사이클 오류 검사까지 구현한 것은 아닙니다.

예시 · 확인할 점

정상: $B \rightarrow A, C \rightarrow A$

순환: $A \rightarrow B \rightarrow A$

A가 먼저여야 B를 출력하고,
B가 먼저여야 A를 출력하는 모순.

방문 집합 없는 탐색은 반복 위험.
현재 ANCESTORS는 set으로
재방문을 건너뛵니다.

DAG여야 모든 부모를 먼저 출력하는 순서를 만들 수 있습니다.

남은 부모가 0개이면 출력합니다

평가자 질문

LOG에서 “부모가 먼저” 조건을 만족시키기 위해 어떤 접근(예: 위상 정렬 성격의 출력)을 적용했는지 설명할 수 있는가?

쉬운 답변 · 현재 구현

Kahn 알고리즘은 각 커밋의 남은 부모 수를 셉니다. 부모가 0개인 커밋을 큐에 넣고, 하나를 출력하면 그 자식의 남은 부모 수를 줄입니다. 0이 된 자식을 큐에 추가합니다.

저장 참조는 자식 → 부모지만, 계산할 때는 부모 → 자식 목록을 만듭니다.
전체 작업량은 $O(V+E)$ 입니다.

예시 · 확인할 점

$A \rightarrow B, A \rightarrow C$ 로 계산

시작: $A=0, B=1, C=1$

A 출력: $B=0, C=0$

다음: B와 C 출력

V = 커밋 수

E = 부모 연결 수

큐는 인덱스로 읽어

$\text{pop}(0)$ 의 이동 비용을 피합니다.

이미 출력한 부모를 하나씩 지워, 모든 부모가 끝난 자식만 출력합니다.

BFS는 가까운 커밋부터 확인합니다

평가자 질문

PATH에서 최단 경로를 찾기 위해 어떤 알고리즘(예: BFS)을 선택했고, 간선을 무방향으로 정의한 이유를 설명할 수 있는가?

쉬운 답변 · 현재 구현

연결 하나를 지나갈 때 비용이 모두 1이므로, 가까운 곳부터 탐색하는 BFS가 최소 간선 수를 찾습니다.

과제에서 PATH의 부모 연결을 무방향으로 보라고 정했기 때문에 양쪽으로 이동합니다. 이는 저장된 부모 참조 방향과 구분해야 합니다.

예시 · 확인할 점

$B \rightarrow A, C \rightarrow A$ 가 저장돼 있어도
PATH에서는 $B \leftrightarrow A \leftrightarrow C$ 입니다.

B에서 시작

거리 0: B

거리 1: A

거리 2: C

거리 동물이면 양쪽 BFS 결과를
이용해 hash 사전순 경로 선택.

같은 비용의 연결에서는 BFS가 가장 적은 연결을 지나는 길을 찾습니다.

병합 정렬은 평균·최악 $O(N \log N)$

평가자 질문

정렬 알고리즘의 평균/최악 시간복잡도와 안정 정렬 여부를 설명할 수 있는가?

쉬운 답변 · 현재 구현

데이터를 절반씩 나누면 약 $\log_2 N$ 개 층이 생깁니다. 각 병합 층에서는 전체 N 개 데이터를 처리하므로 평균·최악 시간복잡도가 $O(N \log N)$ 입니다. 추가 공간은 $O(N)$ 입니다.

비교 키가 같을 때 $\leq$ 로 왼쪽 원소를 먼저 선택하므로 안정 정렬입니다. 원래 입력은 바꾸지 않고 새 결과 목록을 반환합니다.

예시 · 확인할 점

$8 \rightarrow 4 \rightarrow 2 \rightarrow 1$: 분할 3단계
층마다 전체 약 8개 처리

정렬 전
Bob, Alice①, Alice②
정렬 후
Alice①, Alice②, Bob

①·②는 설명용 표식.
같은 이름의 원래 순서는 유지.

절반씩 나누고 합치며, 같은 값끼리는 원래 순서를 유지합니다.

색인은 검색어로 바로 찾는 목록입니다

평가자 질문

역색인이 순회 검색보다 빠른 이유를, 자료구조/시간복잡도 관점에서 설명할 수 있는가?

쉬운 답변 · 현재 구현

전체 순회는 커밋 N개를 하나씩 확인합니다. 역색인은 단어 또는 작성자에 연결된 hash 목록을 dict에서 평균 $O(1)$ 에 찾고, 결과 k개를 복사·출력하는데 $O(k)$ 가 듭니다.

메시지 검사 비용을 빼더라도 순회는 $O(N)$ 이고 실제 비용은 메시지 길이에 도 영향을 받습니다. 여러 단어의 구문 검색은 후보 메시지를 추가 검사하므로 항상 $O(1)$ 은 아닙니다.

예시 · 확인할 점

단어 색인

login → [B]

feature → [B, C]

작성자 색인

Alice Kim → [A, B, C]

커밋 100만 개, 결과 3개라면
목록을 찾고 3개 결과만 처리.
검색어 정규화 비용은 별도입니다.

미리 만든 목록으로 검색 범위를 줄이지만 결과 출력 비용은 남습니다.

10배 커지면 전체 탐색이 부담됩니다

평가자 질문

커밋 수가 10배 늘어났을 때 병목이 될 지점을 예측하고, 개선 방향(자료구조/알고리즘)을 설명할 수 있는가?

쉬운 답변 · 개선 제안(미구현)

현재 LOG·PATH·조상 탐색은 방문하는 커밋과 연결 수에 비례합니다. PATH는 매 요청마다 양방향 연결 목록도 다시 만듭니다. 정렬은 $O(N \log N)$, 큰 검색 결과의 복사·출력도 병목이 될 수 있습니다.

개선안은 인접 목록을 커밋 생성 시 갱신하기, 변경 시 무효화하는 캐시, 결과 페이지 나누기, hash 길이 늘리기입니다. 개선 효과는 실행 시간·메모리를 측정해 판단합니다.

예시 · 확인할 점

현재 구현이 아닌 개선 제안

N 이 10배, 연결도 10배라면 $O(V+E)$ 작업은 대략 10배. 정렬은 정확히 10배가 아니라 $10N \cdot \log(10N)$ 에 비례.

공통 단어 검색 → 결과도 커짐.
6자리 hash → 충돌 부담 증가.

먼저 실제 병목을 측정하고, 반복 계산과 큰 출력을 줄이겠습니다.

부모 방향만 허용하면 길이 줄어듭니다

평가자 질문

PATH의 간선 정의를 “부모 방향만 허용”으로 바꾸면 결과가 어떻게 달라지고, 구현은 무엇을 바꿔야 하는지 설명할 수 있는가?

쉬운 답변 · 개선 제안(미구현)

방향을 자식 → 부모로 제한하면 출발 커밋의 조상 쪽으로만 이동할 수 있습니다. 형제 커밋 사이에 공통 부모가 있어도 부모에서 자식으로 내려갈 수 없어 경로가 없어질 수 있습니다.

인접 목록에는 `adjacency[child].append(parent)`만 둡니다. 출발점은 부모 방향으로 BFS, 도착점은 반대 그래프로 BFS해 남은 거리를 계산해야 합니다. 자기 자신은 여전히 거리 0입니다.

예시 · 확인할 점

현재: $B \leftrightarrow A \leftrightarrow C$
 PATH B C → $B \rightarrow A \rightarrow C$

부모 방향만: $B \rightarrow A, C \rightarrow A$
 PATH B C → No path
 PATH B A → $B \rightarrow A$
 PATH A B → No path

개선안: 두 번째 BFS에는 간선을 뒤집은 그래프를 사용.

연결돼 있어도 화살표 방향으로 갈 수 없으면 경로가 없습니다.

부모 우선과 작성자순을 함께 지키려면

평가자 질문

LOG --sort-by=author 요구사항이 “부모-자식 선호도 유지”로 강화된다면, 어떤 전략으로 해결할지 설명할 수 있는가?

쉬운 답변 · 개선 제안(미구현)

먼저 위상 정렬로 지금 출력해도 되는 후보를 정하고, 후보 중 작성자 이름이 가장 앞선 커밋을 선택합니다. 선택 후 자식의 남은 부모 수를 줄여 새 후보를 추가합니다.

모든 커밋을 작성자 이름만으로 정렬하면 부모보다 자식이 먼저 나올 수 있습니다. 두 조건이 충돌하면 부모 우선을 지키고, 선택 가능한 후보 안에서 작성자순을 적용해야 합니다.

예시 · 확인할 점

현재 구현이 아닌 강화 전략

부모 Zed → 자식 Alice
전체 이름순: Alice → Zed (위반)
강화 규칙: Zed → Alice

동시에 가능한 Alice와 Bob은
Alice를 먼저 선택.

직접 만든 최소 힙이면
 $O(E + V \log V)$ 로 후보 관리.

부모 조건을 먼저 지키고, 준비된 후보들 안에서 작성자순을 고릅니다.

순번과 난수는 재현성이 다릅니다

평가자 질문

해시 생성 방식을 카운터 기반 ↔ 난수 기반으로 바꿀 때 테스트/재현성/디버깅에 미치는 영향을 설명할 수 있는가?

쉬운 답변 · 개선 제안(미구현)

카운터는 0, 1, 2처럼 증가하므로 입력 시각과 시작 상태까지 고정하면 생성 과정을 재현하기 쉽습니다. 현재 hash에는 시각도 들어가므로 카운터만으로 실행마다 같은 hash가 나오지는 않습니다.

난수는 실행마다 달라져 재현이 어려워질 수 있습니다. 테스트에서는 난수 생성기를 주입하고 seed·시각을 고정하거나 지정 값을 반환하게 합니다. 어느 방식이든 기존 hash 검사와 충돌 재시도가 필요합니다.

예시 · 확인할 점

현재: 내용 + 시각 + 카운터
변경안: 내용 + 시각 + 난수

테스트
고정 시각·고정 카운터/seed
강제 충돌 → 재시도 확인

디버깅
생성 조건을 기록하면 재현 쉬움.
테스트용 seed는 보안 보장이 아님.

카운터와 난수 모두 중복 검사가 필요하며, 재현 조건을 따로 고정해야 합니다.